

BIM and ReadSeal: Safe Early Reuse of Shared GPU Tensors in Autonomous-Driving Middleware

Xingbo Hui, Xuetao Jiang, Guodong Ye, Ruochen Shao, Tianyi Wang, Sicheng Wu, Rui Zhou*, Qingguo Zhou*
School of Information Science and Engineering, Lanzhou University, Lanzhou, China

*Co-corresponding authors: zr@lzu.edu.cn, zhouqg@lzu.edu.cn

Abstract—Autonomous-driving processes share large GPU tensors to avoid per-recipient copies. When may the producer overwrite that shared buffer with the next frame? Waiting only for submitted GPU work can miss a reader that has not started; waiting for every result can leave the GPU idle while new frames are refused. Our key insight separates two lifetimes: results retain a frame’s identity until delivery, but its storage is needed only until the last read completes. BEV-in-Memory (BIM) makes this distinction explicit through per-recipient borrows. ReadSeal derives the last-read boundary by tracking which tensors share the input’s storage, registers readers before any submits GPU work, and checks that the analyzed program matches execution. Subsequent computation uses private intermediate results while the shared buffer serves the next frame. In an A100 pipeline with four recipients and early source reads, one BIM slot delivers $1.31\text{--}1.50\times$ the correct, timely results of one-slot full retention at 25–40 Hz. Mean latency increases by 8–11 ms and the per-run 95th percentile by 15–19 ms. ReadSeal achieves the performance of maintained hand-placed boundaries while automatically re-deriving boundaries after model changes and rejecting stale plans. Ordered tests expose wrong-frame reads under stream-only release, and model-update tests validate ReadSeal’s derived boundaries. Together, BIM and ReadSeal turn the last source read into an automatically maintained point for safe cross-process reuse.

Index Terms—Autonomous driving, ROS 2, GPU memory sharing, zero-copy IPC, buffer lifetime, alias analysis.

I. INTRODUCTION

Consider a driving stack built on Autoware or another framework based on ROS 2, the Robot Operating System [1], [2]. A camera–LiDAR fusion node publishes a bird’s-eye-view (BEV) feature map, a top-down grid of features around the vehicle. Separate processes for 3D detection, tracking, motion planning, and data recording read it on their own schedules (Fig. 1a); BEVFusion itself feeds one BEV map to both a detection and a map-segmentation head [3]. We call these processes *recipients*. Our BEV map is a $1\times 512\times 180\times 180$ fp32 tensor of 63.3 MiB [3]; private copies for four recipients would add 256 MiB of device memory (64 MiB each after allocator rounding), a cost that grows with every recipient and resident frame and matters on automotive systems-on-chip whose GPU shares one DRAM with the rest of the stack [4]. Zero-copy middleware lets recipients map the sender’s buffer instead [5], [6], and recent ROS 2 facilities extend this to GPU memory [7], [8]. Sharing raises a question that copying never had to answer: when may the producer overwrite the shared buffer, which we call a *slot*, with the next frame?

The two obvious answers fail (Section II). *Stream-only release* reuses the slot once all GPU work submitted so far has finished. A recipient that has not yet submitted its reads, such as one still waiting in its queue, then silently computes on the next frame: a detector reports objects seen in frame $k+1$ under the timestamp of frame k , and at 20 Hz a vehicle driving at 20 m/s moves a full meter between the two. *Full retention* holds the slot until every result is ready, as reference-counted zero-copy transports do when each recipient keeps its reference until done [5], [6]. It is safe, but a frame that finds the slot held is dropped: at 20 Hz, full retention drops every other frame while the recipients sit idle (Fig. 2). Releasing at a hand-placed point after the last read avoids both failures, but only until a model edit moves that read.

Key insight. A shared tensor, the *source*, has two lifetimes (Fig. 1b). Its *storage lifetime* ends when the last operation that reads it completes; its *provenance lifetime* lasts until every result computed from it is delivered, because those results carry the frame’s identity. The two can differ widely: in the TransFusion detection head that reads our BEV map, only the first of 287 operations reads it. Later operations use intermediate results in private storage, so they can continue after the shared input is overwritten. A library loan works the same way: a student returns the book once she has read it, although her essay still cites that edition. Like her loan, each recipient’s *borrow* on the slot lasts only while it may still read the source. Ending the borrow at the last read lets the slot take the next frame while the previous frame’s computation continues.

Ending the borrow at the last read is safe only if three questions have answers. (1) Which operation reads last? *Views*, tensors that share the input’s storage, extend its reads, and one model edit can move this *read boundary* from the first operation to the second-to-last (Fig. 3). (2) Who may still read? A reader that has not yet submitted GPU work leaves nothing to wait on. (3) Is the analyzed program the one that runs? A boundary derived for one model version can be wrong for the next.

We present BEV-in-Memory (BIM), a middleware contract built on this distinction, and ReadSeal, its mechanism for answering the three questions. BIM keeps apart a frame’s identity, a recipient’s view of its bytes, and the recipient’s borrow of the slot, and registers every recipient’s borrow when a frame is published, before any recipient starts (Section III). ReadSeal answers the three questions in turn: it derives each read boundary by alias analysis of the model graph, enrolls

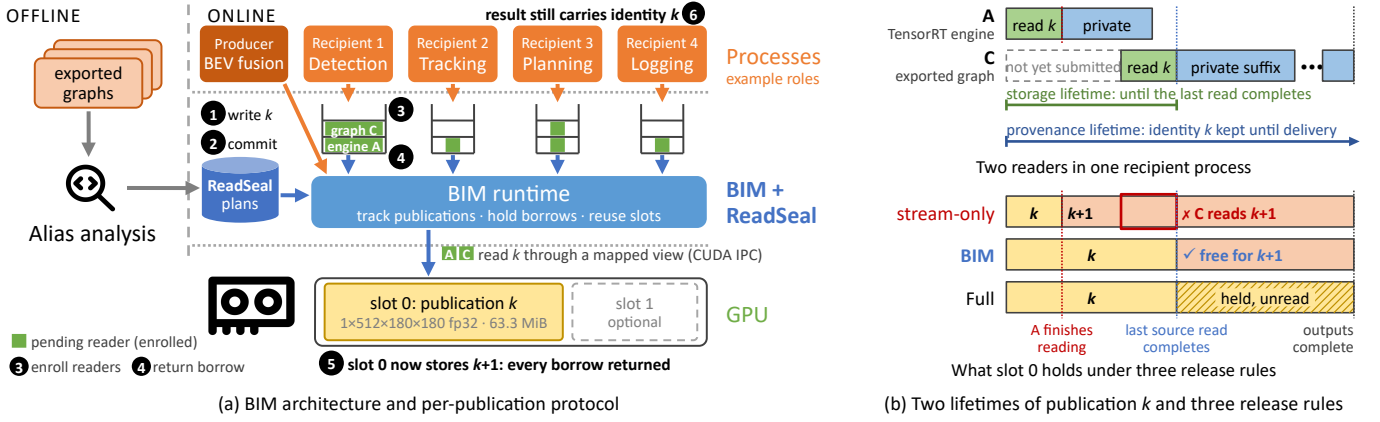


Fig. 1. **BIM at a glance.** (a) Offline, ReadSeal analyzes each exported model graph into a plan. Online, the producer writes frame k into a slot and commits it (1, 2); each recipient enrolls its readers before they submit GPU work (3) and returns its borrow once their source reads complete (4); the producer may overwrite the slot after every borrow is returned (5), while results still carry identity k (6). (b) Time runs left to right; TensorRT engine A and exported graph C are the readers of the last recipient holding a borrow. Stream-only release lets $k+1$ overwrite the slot before the graph reads it; full retention holds the slot until the outputs complete; BIM frees it at the last source read.

each recipient’s readers before they submit work (*enrollment*), and binds each plan to the code, model state, and input layout that execute (*binding*; Section IV).

The throughput benefit comes from ending the borrow at the last source read; a carefully maintained hand-placed boundary obtains the same benefit. ReadSeal automates that maintenance: it derives the boundary from each model version and rejects execution when the code, model state, or input no longer matches the plan. Our A100 evaluation separates these contributions by measuring delivery under load, then testing boundary derivation and checks under model changes. Our contributions are:

- **The two-lifetime view of a shared GPU tensor** (Section II). It explains two failures, a silent wrong-frame read caused by composing two correct mechanisms (stream-ordered reuse and GPU events) and the GPU that full retention leaves idle, and shows that release rules are also admission policies, deciding which frames enter the pipeline.
- **BIM and ReadSeal** (Sections III–IV), a cross-process borrowing protocol and graph analysis that derive each last-read boundary, register pending readers, and check that the analyzed program matches execution.
- **A measured account of reuse and admission** (Section V), connecting slot hold time to timely delivery and queuing latency, together with model-update tests of automatically maintained boundaries.

II. BACKGROUND AND MOTIVATION

GPU background. GPU work is asynchronous: a process enqueues *kernels*, the functions the GPU runs, on a *stream*, a queue the GPU executes in order, and moves on. A read is thus over only when its kernel has completed. An *event* is a marker in a stream that completes once the work enqueued before it has finished. PyTorch’s `record_stream` lets the caching allocator reuse a buffer only after the work already enqueued on the recorded streams has completed [9]. CUDA

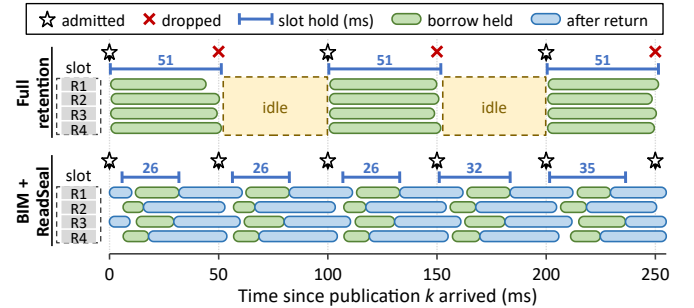


Fig. 2. **Full retention refuses frames while the GPU idles.** Measured one-slot trace at 20Hz (repetition 0); R1–R4 are the recipients, and labels give slot hold times in ms. Full’s 51 ms hold just exceeds the 50 ms frame period, so every other frame is refused although the recipients sit idle: full retention is not work-conserving. Under BIM, each recipient’s work after its borrow return overlaps the next frame.

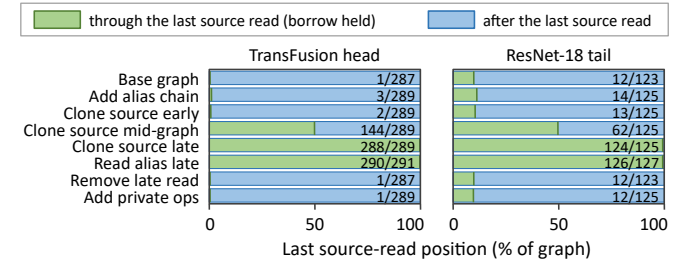


Fig. 3. **Model edits move the last source read.** Each bar is a graph snapshot; green marks the operations up to and including the last one that may read the shared source, over which the recipient holds its borrow. Each snapshot applies one edit to the base graph (Remove late read undoes Clone source late); labels give the last source-reading operation/total operations.

inter-process communication (IPC) lets another process map the same device buffer and wait on shared events. But an event can mark only work already enqueued: no process can wait on reads that another has not yet submitted.

Releasing too early fails silently. The simplest case is a recipient still waiting in its queue. It has enqueued no GPU

TABLE I

RELEASE OPTIONS FOR FOUR RECIPIENTS OF ONE BEV SOURCE (K : CAP ON OUTSTANDING RECIPIENT REQUESTS). [†] SILENTLY, UNLESS DIGESTS ARE REFRESHED BY HAND; READSEAL REJECTS A STALE PLAN. [‡] MISSES READERS THAT HAVE NOT YET SUBMITTED WORK.

Policy	Extra memory	Slot reusable after	Read boundary	Wrong frame?
Stream-only	—	submitted reads	stream idle	possible [‡]
Full retention	—	all outputs	—	no
Full, 2 slots, $K=8$	one slot	all outputs	—	no
Clone-on-accept	256 MiB	all copies	fixed	no
Manual	—	all last reads	by hand	if stale [†]
BIM + ReadSeal	—	all last reads	derived	no

work, so a rule that waits only for submitted work, such as `record_stream` or a read event, returns the slot once the other recipients’ reads complete. The producer writes $k+1$, and the waiting recipient later computes a numerically valid result on it while its request still names k . No crash or error signals the failure, so a downstream planner has no reason to distrust the result. The same gap can open inside a recipient that combines a TensorRT engine with a framework graph. In our running example (Fig. 1b), the two *readers* are TensorRT engine A, which signals an input-consumed event after reading its input [10], and exported graph C. An adapter forwards a view to the graph without reading the source itself. Once the engine’s event completes, stream-only release returns the slot although the graph has not submitted its reads. Section V-E tests this ordering. ROS 2’s CUDA buffer backend exposes the same choice: a subscriber’s read handle records a read event when destroyed, and the block is recycled once every handle is released [11]; destroying the handle after the engine’s read is stream-only release, and holding it until the outputs complete is full retention.

Releasing too late wastes the GPU. Full retention’s 51 ms hold exceeds the 50 ms period at 20 Hz, so every other frame is refused while the GPU idles (Fig. 2). Section V-C compares early reuse with adding a second slot.

Hand-placed read boundaries go stale. The read boundary depends on the recipient’s whole graph: a view can carry a source read into later operations (Fig. 3). In our model-update tests, 14 constructed graph edits, a batch-normalization folding, and a public model substitution move 13 boundaries across 16 transitions (Section V-E). A hand-placed boundary must follow these changes to avoid a wrong-frame read. Table I summarizes the options. BIM tracks every recipient before any runs, and ReadSeal derives and checks the boundary against the executing program.

III. BIM: A PUBLICATION-AWARE CONTRACT

BIM keeps apart three things that zero-copy sharing usually conflates: memory, identity, and permission to read. A *slot* is memory: one of N reusable GPU buffers in the producer’s pool on one machine. A *publication* is one frame’s tensor written into a slot, with an identity that every result computed from it carries; the same slot holds publication k and later $k+1$. Its tensor, as recipients read it, is the *source*. A *recipient* is

a process with one or more *readers*, such as engine A and graph C, which access the source through a *view* that fixes how its bytes are interpreted. A recipient’s *borrow* is its permission to read the slot, held until none of its readers, running or not yet submitted, can still read the source. Borrows decide when memory is reused; identities decide how results are labeled.

Descriptors and views. Recipients receive small descriptors over a control plane such as ROS 2 and map the resident tensor through a backend, here CUDA IPC via PyTorch’s tensor-sharing reductions [12]. A descriptor carries a publication identity (producer instance and sequence number), a storage locator, and an interpretation: element type, shape, strides, and a checked view range. The view specifies which bytes a recipient may address; the identity specifies which frame those bytes belong to. BIM rejects stale identities and incompatible interpretations.

Borrow bits and admission. A process-shared lock protects slot reservation, the publication sequence, and one borrow bit per recipient, set while that recipient may still read the slot. An arriving frame takes a slot whose bits are all clear or, if none is free, is dropped, not blocked. Once the frame is written, the producer *commits* the publication: it sets every recipient’s bit before sending any descriptor, so a recipient is covered even while it waits in its queue. A recipient clears its bit when it returns its borrow; clearing the last bit makes the slot available again without freeing or unmapping it. If publication k is committed at time c_k and recipient j returns its borrow at time $t_{j,k}$, the slot is held for H_k , and with one slot a frame arriving at time a is admitted only if

$$a \geq c_k + H_k, \quad H_k = \max_j t_{j,k} - c_k. \quad (1)$$

The release rule therefore also decides admission: full retention sets $t_{j,k}$ to the time j ’s outputs complete, and ReadSeal to the time j ’s last source read completes (Section IV-B). An optional cap K bounds outstanding recipient requests, each counted from admission to receipt at the output sink. Admission thus has two gates, a free slot and the cap: under overload, BIM refuses frames rather than overwrite a slot that may still be read. ReadSeal enrolls the readers inside each recipient and decides where each borrow ends.

IV. READSEAL: CHECKED BORROWING

ReadSeal works in two phases (Algorithm 1). Offline, once per model version, it turns each graph a recipient runs into a *plan* with one part per question of Section I. The first splits the graph at its read boundary into a *source-reading prefix*, ending with the last operation that may read the source, and a *private suffix*, which computes the rest of the frame’s results without reading the shared storage (Section IV-A). The second lists the *readers* to enroll, the components inside the recipient that read the source, such as engine A and graph C (Section IV-B). The third holds *binding tokens*, identifiers of the code, model state, and input layout the plan was derived for (Section IV-C). Online, an *invocation* is one run of a plan on one publication; its recipient may return its borrow only after every operation

Algorithm 1 BIM’s commit and ReadSeal’s plan and invocation

```

1: procedure PUBLISH(frame  $k$ ) ▷ producer (BIM)
2:   if no slot is free or  $K$  requests are outstanding, drop  $k$ 
3:   write  $k$  into a free slot  $s$ ; commit: set  $b_{s,j}$  for all  $j$ 
4:   send each recipient a descriptor of  $(k, s)$ 
5: procedure BUILDPLAN(graph  $G$ ) ▷ offline, per model version
6:   compute  $Q_r$  by (2) and  $\beta$  by (3)
7:   split  $G$  at  $\beta$ ; record readers  $U$  and tokens  $T$ 
8: procedure INVOKE(plan, publication  $k$ ) ▷ recipient  $j$ 
9:   check  $T$ ; keep identity  $k$  for every output
10:  enroll every reader in  $U$ , before any submits GPU work
11:  submit the stages in declared order, checking  $T$  at each entry
12:  on failed check: cancel unsubmitted reads, drain, publish nothing
13:  on each completed or cancelled read: if (4) holds, clear  $b_{s,j}$ 
14:  on completed outputs: publish them with identity  $k$ 

```

that may read the source has completed or was cancelled before starting.

A. Deriving the Read Boundary

The analysis runs on each graph, a fixed-shape, functional directed acyclic graph (DAG) of operations. Of the two lifetimes, only storage needs analysis. Every result carries the frame identity attached at acceptance. Storage sharing propagates through views: a slice of the source is a view, so an operation on the slice still reads the source, whereas a convolution reads the source but writes fresh storage, so operations on its output do not. In our running example, the adapter only forwards a view; the graph receiving that view reads the source.

Formally, let $S(v)$ be the set of shared roots whose storage a value v may alias, with $S(r) = \{r\}$ for the source r . For an operation o with inputs v_i and output v_o , let $\mathcal{A}_o$ index the inputs the output may alias and $\mathcal{R}_o$ the inputs o reads. Then

$$S(v_o) = \bigcup_{i \in \mathcal{A}_o} S(v_i), \quad (2)$$

$$Q_r = \{o : \exists i \in \mathcal{R}_o, r \in S(v_i)\},$$

where Q_r is the set of operations that may read the source. With the operations numbered $o_1, \dots, o_n$ in execution order, the read boundary is the position of the last one in Q_r ,

$$\beta = \max\{i : o_i \in Q_r\}, \quad (3)$$

so the source-reading prefix is $o_1, \dots, o_\beta$ and the private suffix $o_{\beta+1}, \dots, o_n$. Fig. 3 labels each graph snapshot with β/n ; the base head has $\beta = 1$ and $n = 287$.

Operator summaries cover 42 reviewed operator variants in ATen, PyTorch’s operator library, plus selection of a tuple element at a fixed index. Each summary counts every operand as read and treats an output as sharing storage whenever the operator’s declared signature permits aliasing with an input. Plan construction rejects unsupported mutation, opaque callbacks, and source views that escape the graph. The implementation compiles the prefix and suffix into separate executable stages. A CUDA event after the prefix then marks completion of all its source reads.

B. Enrolling Readers Before Submission

Registration happens at two levels. BIM registers recipients: its commit sets one borrow bit per recipient (Section III).

ReadSeal registers the readers inside a recipient: when it accepts an invocation, it enrolls every reader in the plan at once, before any of them submits GPU work. Each reader u marks the end of its source reads with one completion event, A’s input-consumed event or the event recorded after C’s prefix. Let U_j be the readers of recipient j . Then j returns its borrow for publication k at the first moment when

$$\forall u \in U_j : \text{done}_u \vee \text{cancelled}_u, \quad (4)$$

where done_u holds once u ’s completion event has completed and cancelled_u once u has been cancelled before submission. The condition ranges over every enrolled reader, not only those with work in flight: when A’s event completes, C’s read is still pending, so the bit stays set although C has enqueued no GPU work. Returning the borrow clears only j ’s bit; the slot takes the next publication once every recipient has returned its borrow, while j ’s private suffix keeps computing results that carry identity k . Enrollment prevents the wrong-frame read of Section II; the analysis sets how early the borrow ends.

C. Binding Plans to Execution

At construction, ReadSeal captures the functions to run, clones model tensors, and records binding tokens T : identifiers and metadata for code and globals, model storage and version, input layout, and the compiled stages. Acceptance and each stage entry compare T with the corresponding values of what is about to run, $\hat{T}$, and proceed only if $\hat{T} = T$ (Algorithm 1). A failed check cancels unsubmitted reads, drains started work, and publishes nothing. The application supplies the exported graph, the completion event that marks each native engine’s last input read, and the component and output dependencies; new operators require reviewed summaries.

D. Correctness Conditions

The analysis must include every operation that can read the source. It assumes that operator summaries conservatively describe reads and storage sharing, kernels obey those summaries, and the graph executes in the analyzed order.

Proposition 1. *Under these assumptions, Q_r in Eq. (2) contains every operation that can read the shared source. No operation after the derived boundary β reads that source.*

Proof sketch. Initially $S(r) = \{r\}$. At each operation, the union in Eq. (2) preserves the shared roots of every input that its output may alias. Induction over graph order therefore includes every source alias in the corresponding $S(v)$. The read rule places every operation consuming such storage in Q_r ; by Eq. (3), none follows o_β . $\square$

For the runtime, each reader’s completion event must occur after its source reads, mappings must remain valid, and identities must remain unique while referenced. Algorithm 1 registers every reader before submission, checks the plan at execution, and returns a borrow only when Eq. (4) holds. BIM permits overwrite after all recipient bits clear. Together, these steps preserve the source until every reader finishes, while results keep the identity captured at acceptance.

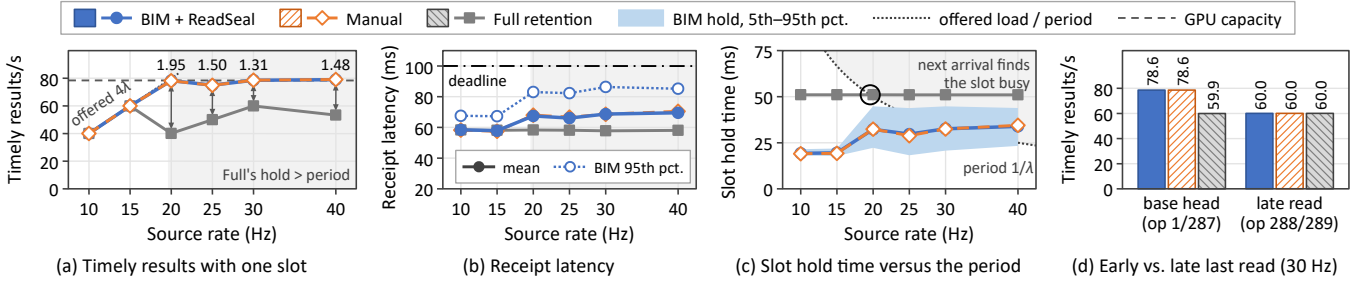


Fig. 4. **Early reuse raises one-slot delivery under load, with a latency cost.** One slot, no pipeline-wide cap K ; six runs per condition and policy. Each input frame offers four recipient results; results/s counts individual results. (a) Labels give BIM/Full ratios. (b) Mean receipt latency; BIM’s 95th percentile is computed per run and averaged. (c) Mean hold and BIM’s 5th–95th percentile; shading marks where Full’s hold exceeds the frame period. (d) Moving the last read to the graph’s end removes the gain.

V. EVALUATION

The evaluation answers five questions in turn: when early reuse pays and what it costs in latency (Section V-B, Fig. 4); whether a second slot could do the same (Section V-C, Fig. 5); how jitter, mixed recipients, and copying change the picture (Section V-D, Fig. 6); whether BIM stays safe under reordering and model change (Section V-E); and what ReadSeal’s checks cost (Section V-F).

A. Setup

Scope and platform. We evaluate cross-process reuse of saved BEV features on one NVIDIA A100 PCIe 40GB GPU. The producer, four recipients, and output sink communicate through multiprocessing queues and share storage and events through CUDA IPC. This setup measures reuse and admission independently of feature extraction; it does not run ROS 2, the BEV front end, or an embedded driving stack. The runtime uses PyTorch 2.1.2, CUDA 12.1, and TensorRT 10.3. Without NVIDIA’s Multi-Process Service (MPS), recipient kernels take turns on the GPU. Records, validators, campaign details, and analysis scripts are public.¹

Workload. The exported graph (C) is BEVFusion’s functional TransFusion head [3], [13], with 287 operations. The producer publishes precomputed 63.3 MiB feature maps from three fixed nuScenes-mini frames [14]. The native engine (A) runs the head’s shared convolution, ReLU, and global average pooling in TensorRT; an adapter forwards a view to the graph. This controlled composition gives each recipient two readers of the same source, each with its own completion event. Four recipients run identical compositions in the rate and slot sweeps; Section V-D varies their components. A pretrained ResNet-18 tail, comprising residual stages and classifier with saved stem activations as the shared source, serves in the correctness, model-change, and cost tests.

Policies. All policies share the pool, workloads, queues, and admission settings. BIM uses ReadSeal; *Full* retains the source until its outputs complete. The hand-placed baseline has two implementations: *Manual*, used in the rate and slot sweeps, independently enrolls both readers and checks hand-maintained code and model digests; *Manual-checked*, used in the jitter

and mixed-recipient tests, uses BIM’s runtime and binding checks with a hand-placed boundary. *Clone-on-accept* gives each recipient a private copy when it accepts a request.

Metrics. The source publishes frames at rate λ , in Hz. Each published frame offers one *result*, the output delivered by one recipient, to each of four recipients; offered load is therefore 4λ results/s. A result is *timely* if it reaches the sink after device-to-host transfer and IPC, passes source-identity and reference-output checks, and arrives within 100 ms of the frame’s planned arrival. Late results and refused frames are counted separately. *Receipt latency* runs from planned arrival to sink receipt; *slot hold time* runs from commit to the last borrow return. Section V-D additionally measures complete frames: all four intended results must be correct and timely, with every scheduled frame in the denominator.

Queue settings. The rate and slot sweeps permit at most two pending requests and four in flight per recipient. Optional cap K limits outstanding recipient requests across the pipeline; $K=8$ accommodates two full groups of four requests. *No pipeline-wide cap* leaves the per-recipient queue limits in place.

Statistics. Rate and slot runs have 2 s of warm-up, 12 s of measured arrivals, and a complete drain. Condition and method order are randomized, with six paired repetitions per condition; brackets give pointwise 95% intervals from 20,000 paired whole-run bootstrap draws. The two campaigns in Section V-D use queues of eight, 10 s of warm-up, and 30 s of measurement, with six paired runs and pointwise 95% t intervals. Campaigns are analyzed separately. Latency summaries describe individual recipient results in the rate sweep (mean and per-run 95th percentile) and complete correct frames in the mixed-recipient study (per-run 99th percentile).

B. Early Reuse Raises Delivery Under Load

Once the frame period falls below full retention’s slot hold time, early reuse delivers more (Fig. 4a). All policies deliver the offered 40 and 60 results/s at 10 and 15 Hz. At 20 Hz, BIM delivers 78.3 results/s and Full 40.1, a paired difference of 38.2 [37.9, 38.5]; at 25–40 Hz, BIM delivers 1.31–1.50 $\times$ as many. BIM stays within 0.2 results/s of Manual at every rate.

Why the gap opens. The four recipients take turns on the GPU, so each takes 48–49 ms from acceptance to output, against about 11 ms in isolation (Section V-F), and the pipeline

¹<https://github.com/assasin831/readseal-artifact/tree/ispa2026-r15>.

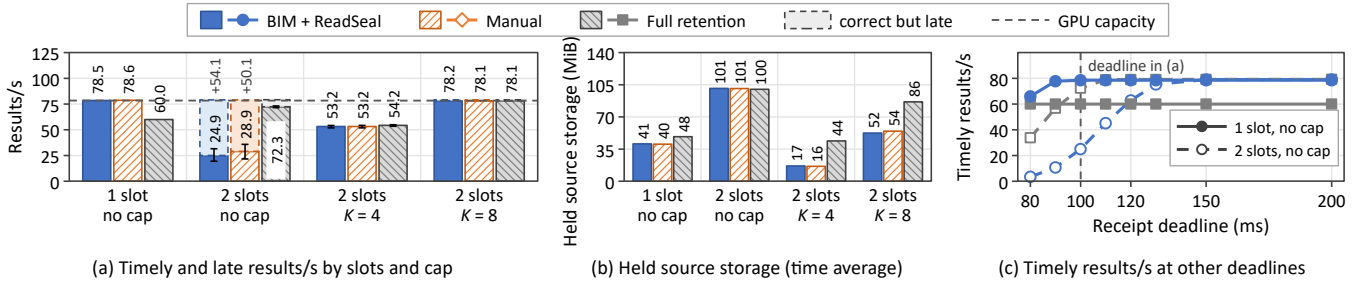


Fig. 5. **One BIM slot matches full retention with two slots and a request cap** (30 Hz; six runs per condition and policy). K counts outstanding recipient requests; eight requests are two full groups of four. (a) Solid: timely results; dashed: correct but late; whiskers: 95% intervals. (b) Time-averaged source storage held by unreturned borrows, not allocated memory. Device peaks are 5,078 MiB with one slot and 5,142 MiB with two for every policy. (c) BIM and Full re-scored at different deadlines using the same runs.

completes about four results per 51 ms, 78.4 results/s (Fig. 4a, dashed). Full’s hold spans nearly this whole time, $H_k = 51$ ms at every rate (Fig. 4c), so by Eq. (1) it refuses the next frame exactly when frames arrive faster than the GPU can finish them: Full admits every other frame at 20 Hz and every third at 40 Hz, and the GPU idles in between (Fig. 2). BIM’s hold ends at the last recipient’s enrolled reads. In the base head these are the first operations, yet the hold lasts 30–34 ms under contention, because the last recipient must wait its turn on the GPU before they run: a read’s position in the graph is not its share of the time. The hold still fits the period at 20 Hz, where BIM refuses only 2% of frames, and BIM reaches the ceiling at 20, 30, and 40 Hz. At 25 Hz, periodic arrivals lock it into admitting three frames in four (75.0 results/s), since a refused frame is replaced only one 40 ms period later. The $1.95\times$ peak arises because Full’s hold barely exceeds the period at 20 Hz; $1.31\text{--}1.50\times$ at 25–40 Hz is the sustained gain.

Latency cost. Early reuse changes admission, not model speed, so the extra results queue behind GPU work (Fig. 4b): BIM’s mean receipt latency at 20–40 Hz is 66–70 ms, 8–11 ms above Full’s, and its per-run 95th percentile averages 82–86 ms against Full’s 67 ms. Re-scored at 100–200 ms deadlines, one-slot BIM’s lead at 30 Hz is unchanged; at a tight 80 ms it narrows from 18.5 to 5.9 results/s (Fig. 5c).

The gain requires a private suffix. In the late-read test at 30 Hz, the base head gives BIM 78.6 results/s and Full 59.9; with the last source read at operation 288 of 289 (Fig. 3’s late clone), all three policies deliver 60.0 (Fig. 4d), because BIM’s hold rises from 32.7 to 51.7 ms and leaves nothing to overlap.

C. One Slot Matches Two, and Two Need a Cap

We first compare one-slot early reuse with a larger pool, then vary admission within that pool (Fig. 5a). At 30 Hz, one-slot BIM delivers 78.5 results/s and one-slot Full 60.0. Adding a second slot raises Full to 72.3 results/s without a pipeline-wide cap; setting $K=8$, enough for two full groups of recipient requests, raises it to 78.1. A tighter cap of $K=4$ yields 54.2. Thus one-slot BIM reaches the measured capacity with 64 MiB less device peak than two-slot Full.

A release rule is also an admission policy. By Eq. (1), a slot pool acts as an admission controller: one slot paces BIM to the GPU by itself, admitting about two thirds of the results

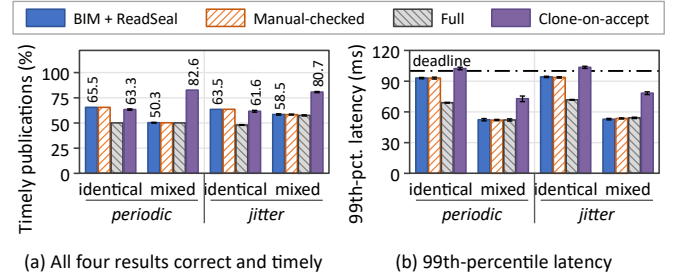


Fig. 6. **Jitter keeps the gain; with mixed recipients, one reading late, copying wins** (30 Hz, one slot, no pipeline-wide cap; 6 runs per bar; whiskers: 95% t intervals). (a) Share of the 900 publications per run whose four results are all correct and timely; (b) per-run 99th-percentile latency of publications whose four results are all correct. *Identical*: four copies of the composition; *mixed*: four different recipients, one reading late.

offered at 30 Hz. Two uncapped slots let BIM admit frames that the GPU cannot start in time; they wait in the recipients’ queues and miss the deadline. Uncapped two-slot BIM admits about as many results as one-slot BIM, all correct, but only 24.9 per second are timely, against 72.3 for Full, whose long hold still throttles admission: the time from a frame’s arrival to the start of its processing rises from about 29 ms under Full to 54 ms. For BIM, already at the GPU’s ceiling, the second slot buys only queueing delay, as in bufferbloat, where oversized network buffers add delay without adding throughput. Manual likewise collapses (28.9), and at a 150 ms deadline uncapped BIM and Full both deliver about 79 results/s (Fig. 5c).

An explicit cap separates admission from safety. At $K=8$, BIM, Manual, and Full all deliver 78.1–78.2 results/s. In the two-slot rate sweep at 20–40 Hz, uncapped BIM delivers 20–40 results/s and Full 71–72, whereas both deliver 78.0–78.3 at $K=8$. Held source storage at $K=8$ (Fig. 5b) falls from 86.4 MiB under Full to 52.3 MiB under BIM at an unchanged device peak. This measures how much source storage is still unavailable for reuse, averaged over time. ReadSeal decides when reuse is safe; the admission cap controls queueing.

D. Jitter, Mixed Recipients, and Copying

The next study tests whether every intended recipient completes on time, using the complete-frame metric in Fig. 6. At 30 Hz with one slot and no pipeline-wide cap, arrivals are

periodic or displaced by up to 0.8 of a period (*jitter*). Recipients are either *identical* copies of the composition or *mixed*: the native engine alone, the exported head alone, their composition, and a composition that reads the source late. Manual-checked supplies the hand-placed comparison in this study.

Jitter keeps the gain. With identical recipients, BIM completes 65.5% of publications with periodic arrivals and 63.5% with jitter, near the two thirds the shared GPU can finish, against Full’s 50.0% and 48.0%. With mixed recipients the slot is held until the late reader’s last read, and BIM completes 50.3% (58.5% with jitter), under one point above Full.

Copying wins when recipients read late. Clone-on-accept frees its slot once the copies are made: with mixed recipients it completes 81–83% of publications, 22–32 points more than BIM, at a 256 MiB higher device-memory peak and 20–25 ms more 99th-percentile latency; with identical recipients it completes about 2 points fewer. A separate 168-run rate sweep with identical recipients isolates the late read: cloning stays within 3 results/s of BIM at every rate but leads by 16.4 (1.27 $\times$) when every recipient reads late. Manual-checked stays within 0.13 percentage points of BIM everywhere, including 12 all-late runs: the gain comes from where the borrow ends, not from how that point is found.

E. Safety Across Reordering and Model Changes

Future readers. In six controlled ordered tests, the exported graph’s outputs under stream-only release matched the replacement frame; BIM’s and Manual’s matched the admitted frame. The graph had not yet submitted its reads when the engine completed, so recording stream use did not cover those later reads [9]. Separately, in the 144 rate and late-read runs, all 109,592 results delivered by BIM, Manual, and Full carried their admitted identity and matched reference outputs.

Boundaries under model change. Fig. 3 shows derived boundaries for sample snapshots; ResNet’s residual connection already needs a later boundary than the head’s. Across eight snapshots per model, three inputs, and two methods, all 96 ordered-overwrite cases (540 output tensors) matched unsplit references after the regenerated boundary, and binding validation rejected all 22 attempted substitutions of plan, code, globals, model state, or input before the affected stage.

Maintenance. We replay 16 transitions (14 constructed graph edits, one batch-normalization folding, and a public ResNet-18 to ResNet-50 substitution) that move 13 read boundaries. Manual must update each by hand and refresh 16 digests even to detect a stale boundary, which otherwise fails silently; ReadSeal re-derives every boundary, without annotations, in 0.08–0.21 s of CPU time per plan. Its checks also catch stale hand-placed boundaries: in the jitter and mixed-recipient tests, a Manual-checked boundary made stale by an added late read was rejected until moved, and all 24 update cases gave exact outputs.

F. Execution Cost and Coverage

An isolated-process test times recipient acceptance to output publication (8 process repetitions of 50 randomized invocations

per method; three inputs; base and late variants of both models). ReadSeal’s checks add 0.8–0.9 ms per invocation, mostly at acceptance and stage submission: 8% over Manual for the base head (11.8 ms against 10.9 ms) and 22–23% for the lighter ResNet tail. In the pipelines above, this cost overlaps shared GPU work; even at 10 and 15 Hz, BIM’s mean receipt latency is within 0.4 ms of Manual’s. A static survey of 14 torchvision tails with untrained weights accepts nine, with boundaries in their first 1–12 operations, and rejects five for unsupported operators.

VI. DISCUSSION

When to use BIM. The measurements connect source reuse to admission: after early source reads, a single slot can pace arrivals to the shared GPU while private computation continues. ReadSeal maintains this boundary as models change. Larger pools need a request cap when queueing threatens deadlines. For late readers, copying trades memory and tail latency for earlier reuse; full retention remains an option when memory rules out copies. A copy is itself a source read, so queued recipients still need to be registered before the slot can be reused.

Portability. BIM requires cross-process storage mapping and completion events that cover each reader’s source access. ROS 2’s CUDA buffer read handles [11] and NvSciStream’s per-consumer packet release [15] provide candidate integration points. A port must implement the storage and event adapter and validate its ordering guarantees. Custom operators require reviewed read and alias summaries; native engines supply an event marking their last input read.

Deployment. The prototype runs declared topologies with single-stream stages; a topology change requires a new plan. A crashed recipient leaves its borrow outstanding and prevents slot reuse. Recovery requires confirmation that the recipient’s GPU work has stopped before its borrow can be removed.

VII. RELATED WORK

Zero-copy robotics middleware. iceoryx and Agnocast give ROS 2 subscribers direct access to publisher-owned shared memory [5], [6]. For GPU data, NITROS keeps its zero-copy path within one process [7], and ROS 2’s `rosidl::Buffer` moves tensor bytes between processes through CUDA IPC [8]. Readers that have not yet submitted, and where to release, are left to the application.

Release fences. Android’s BufferQueue lets a consumer release a pooled buffer with a fence while still reading it [16], and NvSciStream returns a multicast packet to its pool once every consumer has released it [15]. BIM’s borrow bits implement this protocol; ReadSeal derives where the release fence goes, which these interfaces leave to the application.

Last use and safe reclamation. Compilers and runtimes reuse a buffer after its last use, as in TVM’s static memory planning [17], and Rust’s non-lexical lifetimes end a borrow there [18]; all see every use in one program, in program order. Safe memory reclamation, such as read-copy-update (RCU) grace periods [19], reuses memory once no reader can hold it,

but protects only readers that have announced themselves. In RCU terms, BIM’s commit opens every recipient’s read-side section on its behalf, and ReadSeal closes it at the derived last read. ReadSeal’s bindings, like PyTorch 2’s guards [20], reject execution whose assumptions changed.

VIII. CONCLUSION

BIM separates a shared input’s storage lifetime from the identity carried by its results. ReadSeal makes that separation executable: it derives the last-read boundary, registers readers before submission, and checks the plan against the running program. The evaluation shows how safe early reuse improves timely delivery by admitting the next frame while private computation continues, and how model analysis maintains the boundary through updates. The central lesson is that source lifetime is both a correctness condition and an admission decision. Future work will integrate BIM with ROS 2 and automotive sharing backends, measure the checks on embedded cores, and extend the design to concurrent recipients and selective copying for late readers.

REFERENCES

- [1] Autoware Foundation, “Autoware: Open-source software for autonomous driving,” <https://autoware.org/>, 2024.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] Z. Liu *et al.*, “BEVFusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 2774–2781.
- [4] NVIDIA, “CUDA for Tegra application note (CUDA 13.0),” <https://docs.nvidia.com/cuda/archive/13.0.0/cuda-for-tegra-appnote/index.html>, 2025.
- [5] Eclipse Foundation, “Eclipse iceoryx: True zero-copy inter-process communication,” <https://github.com/eclipse-iceoryx/iceoryx>.
- [6] T. Ishikawa-Aso and S. Kato, “ROS 2 Agnocast: Supporting unsized message types for true zero-copy publish/subscribe IPC,” in *Proc. 28th Int. Symp. Real-Time Distrib. Comput. (ISORC)*, 2025, pp. 1–10.
- [7] NVIDIA, “Isaac ROS NITROS: NVIDIA Isaac transport for ROS,” https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_nitros/isaac_ros_nitros/index.html, 2026.
- [8] Open Robotics, “About `rosidl::Buffer` backends (ROS 2 Lyrical documentation),” <https://docs.ros.org/en/lyrical/Concepts/Intermediate/About-Buffer-Backends.html>, 2026.
- [9] PyTorch Contributors, “`torch.Tensor.record_stream` (PyTorch 2.1 documentation),” https://pytorch.org/docs/2.1/generated/torch.Tensor.record_stream.html, 2023.
- [10] NVIDIA, “TensorRT 10.3: Execution context and input-consumed event contracts,” <https://github.com/NVIDIA/TensorRT/blob/release/10.3/include/NvInferRuntime.h>, 2024.
- [11] Open Robotics, “`cuda_buffer_backend` (ROS 2 `rosidl_buffer_backends`),” https://github.com/ros2/rosidl_buffer_backends, 2026.
- [12] PyTorch Contributors, “Multiprocessing package: Sharing CUDA tensors (PyTorch documentation),” <https://docs.pytorch.org/docs/stable/multiprocessing.html>.
- [13] X. Bai *et al.*, “TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1090–1099.
- [14] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 621–11 631.
- [15] NVIDIA, “NvSciStream APIs (NVIDIA DRIVE OS 6.0 Linux SDK API reference),” https://developer.nvidia.com/docs/drive/drive-os/6.0.8/public/drive-os-linux-sdk/api_reference/group_nvsci_stream_apis.html.
- [16] Android Open Source Project, “BufferQueue and Gralloc,” <https://source.android.com/docs/core/graphics/arch-bq-gralloc>.
- [17] T. Chen *et al.*, “TVM: An automated end-to-end optimizing compiler for deep learning,” in *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2018, pp. 578–594.
- [18] N. Matsakis, “RFC 2094: Non-lexical lifetimes,” <https://github.com/rust-lang/rfcs/blob/master/text/2094-nll.md>, 2017.
- [19] P. E. McKenney and J. D. Slingwine, “Read-copy update: Using execution history to solve concurrency problems,” in *Proc. Parallel Distrib. Comput. Syst. (PDCS)*, Oct. 1998, pp. 509–518.
- [20] J. Ansel *et al.*, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, vol. 2, 2024, pp. 929–947.